

Lab 1 – Độ phức tạp và phân tích thuật toán

1. Mục tiêu

Kết thúc bài thực hành, sinh viên có khả năng:

- Nhận diện và ghi được độ phức tạp Big-O cho các đoạn code Python đơn giản (vòng lặp for/while, nested loop).
- Biết đếm sơ bộ số phép toán, giải thích 1–2 câu vì sao lại ra độ phức tạp đó.
- Thấy được sự khác biệt giữa cách cài đặt $O(n^2)$ và $O(n)$ trên cùng một bài toán, và biết cách tối ưu dùng set/dict.

2. Nội dung

Bài 1 – Phân tích độ phức tạp

Trong bài này, sinh viên làm quen với cách:

1. Đọc code Python.
2. Đếm số lần lặp của vòng for/while.
3. Từ đó suy ra Big-O và giải thích 1–2 câu.

Yêu cầu chung cho từng snippet

- Bước 1: Ghi lại đoạn code vào file lab1_bai1.py.
- Bước 2: Thêm comment ngay dưới đoạn code với 2 dòng:
 - Dòng 1: # Độ phức tạp: $O(\dots)$
 - Dòng 2: # Giải thích: ... (1–2 câu, tiếng Việt).

Snippet 1 – Vòng lặp for đơn

FPT POLYTECHNIC

```
def snippet_1(n):
    total = 0      # 1 phép gán
    for i in range(n): # lặp n lần
        total = total + 1
    return total
```

Hướng dẫn phân tích:

- Vòng for chạy n lần vì range(n) tạo ra dãy 0..n-1.
- Mỗi lần lặp có 1 câu lệnh total = total + 1 coi là 1 bước tính toán chính.
- Bỏ qua hằng số, số bước $\approx n \rightarrow$ **Độ phức tạp: $O(n)$** .

Mẫu lời giải sinh viên nên viết:

```
# Độ phức tạp:  $O(n)$ 
# Giải thích: vòng for chạy n lần, mỗi lần chỉ có 1 phép gán/cộng.
```

Snippet 2 – Vòng lặp for lồng nhau

```
def snippet_2(n):
    count = 0
    for i in range(n):
        for j in range(n):
            count += 1
    return count
```

Hướng dẫn phân tích:

- Vòng ngoài: chạy n lần.
- Với mỗi i, vòng trong cũng chạy n lần $\rightarrow$ tổng số lần lệnh count += 1 = $n \times n = n^2$.
- Bỏ hằng số $\rightarrow$ **Độ phức tạp: $O(n^2)$** .

Mẫu comment:

FPT POLYTECHNIC

Độ phức tạp: $O(n^2)$

Giải thích: có 2 vòng for lồng nhau, mỗi vòng chạy n lần $\rightarrow$ tổng $n * n = n^2$ bước.

Snippet 3 – Vòng while chia đôi n

```
def snippet_3(n):
```

```
    steps = 0
```

```
    while n > 0:
```

```
        n = n // 2
```

```
        steps += 1
```

```
    return steps
```

Hướng dẫn phân tích:

- Mỗi lần lặp, n bị chia đôi: $n, n/2, n/4, \dots$ cho đến 1.
- Số lần chia đôi cho đến khi $n \leq 1$ khoảng $\log_2(n)$.
- $\rightarrow$ **Độ phức tạp: $O(\log n)$.**

Mẫu comment:

Độ phức tạp: $O(\log n)$

Giải thích: mỗi vòng while chia n cho 2, nên số vòng $\approx$ số lần chia đôi n về 1 là $\log_2(n)$.

Snippet 4 – Vòng for + hàm gọi $O(1)$

```
def constant_work():
```

```
    x = 1
```

```
    y = 2
```

```
    z = x + y
```

```
    return z
```

```
def snippet_4(n):
```

```
for i in range(n):  
    constant_work()
```

Hướng dẫn phân tích:

- `constant_work()` luôn thực hiện một số thao tác cố định, nên coi là $O(1)$.
- Vòng `for` gọi `constant_work()` n lần $\rightarrow$ tổng thời gian $O(n \times 1) = \mathbf{O(n)}$.

Mẫu comment:

```
# Độ phức tạp:  $O(n)$ 
```

```
# Giải thích: vòng for chạy  $n$  lần, mỗi lần gọi hàm  $O(1)$ , nên tổng thời gian tỉ lệ tuyến tính với  $n$ .
```

Bài 2 – Phân tích độ phức tạp

Trong bài này, mỗi snippet vẫn phải ghi:

- Big-O.
- Giải thích 1–2 câu.

File đề xuất: `lab1_bai2.py`.

Snippet 5 – Vòng for với `range(i)`

```
def snippet_5(n):  
    total = 0  
    for i in range(n):  
        for j in range(i):  
            total += 1  
    return total
```

Gợi ý:

- Khi $i = 0 \rightarrow$ vòng trong chạy 0 lần.

FPT POLYTECHNIC

- Khi $i = 1 \rightarrow$ chạy 1 lần, $i = 2 \rightarrow 2$ lần, ..., $i = n-1 \rightarrow n-1$ lần.
- Tổng số lần lặp là $0 + 1 + 2 + \dots + (n-1) = n(n-1)/2$.
- Sinh viên tự kết luận Big-O và ghi giải thích (kết quả là $O(n^2)$).

Snippet 6 – Vòng while + for

```
def snippet_6(n):
```

```
    k = 1
```

```
    total = 0
```

```
    while k < n:
```

```
        for i in range(n):
```

```
            total += 1
```

```
        k = k * 2
```

```
    return total
```

Gợi ý:

- Vòng while: mỗi lần nhân k với 2 $\rightarrow$ số lần lặp của while là khoảng $\log_2(n)$.
- Mỗi lần while, vòng for bên trong chạy n lần.
- Tổng số bước $\approx$ (số vòng while) $\times$ n. Sinh viên tự suy luận kết quả ($O(n \log n)$) và giải thích.

Snippet 7 – Duyệt list + toán tử in (list)

```
def snippet_7(arr):
```

```
    count = 0
```

```
    for x in arr:
```

```
        if x in arr: # kiểm tra x có trong arr
```

```
            count += 1
```

```
    return count
```

Gợi ý:

- Giả sử arr có n phần tử.
- Vòng for chạy n lần.
- Mỗi lần, phép x in arr phải duyệt cả list $\rightarrow$ tốn $O(n)$.
- Tổng thời gian $\approx n \times n$. Sinh viên tự viết Big-O ($O(n^2)$) và giải thích.

Snippet 8 – Dùng set để tối ưu phép in

```
def snippet_8(arr):
```

```
    s = set(arr)
    count = 0
    for x in arr:
        if x in s:
            count += 1
    return count
```

Gợi ý:

- Tạo set(arr) từ list arr tốn bao nhiêu? (số phần tử = n $\rightarrow O(n)$).
- Vòng for: n lần, mỗi lần x in s là $O(1)$ trung bình.
- Tổng thời gian: $O(n) + n \times O(1)$. Sinh viên tự kết luận ($O(n)$) và giải thích.

Bài 3 – Tối ưu thuật toán từ $O(n^2)$ xuống $O(n)$

Trong bài này, sinh viên sẽ:

1. Phân tích một hàm có độ phức tạp $O(n^2)$.
2. Viết lại hàm mới với độ phức tạp $O(n)$.
3. Chạy thử so sánh thời gian giữa hai hàm với input lớn.

File đề xuất: lab1_bai3.py.

Đề bài

Bài toán: Tìm 2 chỉ số i, j sao cho $\text{arr}[i] + \text{arr}[j] == \text{target}$ ($i < j$). Nếu không có, trả về None.

Đoạn code ban đầu ($O(n^2)$):

```
def two_sum_quadratic(arr, target):  
    n = len(arr)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if arr[i] + arr[j] == target:  
                return (i, j)  
    return None
```

Yêu cầu

1. Bước 1 – Phân tích $O(n^2)$:

- Ghi comment vào code cũ:
 - Độ phức tạp Big-O.
 - Giải thích ngắn (vì sao là $O(n^2)$).

2. Bước 2 – Viết phiên bản $O(n)$:

- Viết hàm mới `two_sum_linear(arr, target)` sử dụng **dict** hoặc **set** để lưu các phần tử đã đi qua.
- Gợi ý ý tưởng:
 - Duyệt từ trái sang phải, với mỗi `arr[i]`, tính `complement = target - arr[i]`.
 - Kiểm tra `complement` đã xuất hiện trước đó chưa (trong dict/set).
 - Nếu có $\rightarrow$ trả về cặp chỉ số.

- Nếu chưa → thêm `arr[i]` vào dict/set và tiếp tục.

3. Bước 3 – So sánh thời gian chạy (tự chọn):

- Tạo một list lớn (ví dụ `n = 50_000` hoặc `100_000`).
- Dùng `time.time()` hoặc `time.perf_counter()` để đo thời gian:
 - Thời gian chạy `two_sum_quadratic` (có thể phải dùng `n` nhỏ hơn nếu quá chậm).
 - Thời gian chạy `two_sum_linear`.
- In ra màn hình để thấy sự khác biệt.

Gợi ý skeleton cho phiên bản $O(n)$:

```
def two_sum_linear(arr, target):  
    seen = {} # key: giá trị, value: chỉ số  
    for i in range(len(arr)):  
        complement = target - arr[i]  
        # TODO: kiểm tra complement có trong seen không  
        # nếu có: trả về (seen[complement], i)  
        # nếu không: thêm arr[i] vào seen  
    return None
```

Gợi ý test và đo thời gian:

```
import time  
import random  
  
arr = list(range(100000))  
random.shuffle(arr)  
target = arr[123] + arr[9876]
```


FPT POLYTECHNIC

```
start = time.time()
print(two_sum_linear(arr, target))
print("O(n) time:", time.time() - start)
```

Với bản $O(n^2)$ chỉ nên test với n nhỏ (vd: 5000) để tránh đợi quá lâu.

Sinh viên tự hoàn thiện code, chạy thử, chụp màn hình kết quả hoặc viết log trong file `.py`.